TEMPORADA XIV

CI NICARAGUA · 2026

DOCUMENTO DE LECTURA

06

// Unidad II · Desarrollo Backend con Java

INTRODUCCIÓN A SPRING BOOT

DE LA CONSOLA A TU PRIMERA API

EN ESTA LECTURA

01 Backend, API, HTTP y JSON
02 ¿Qué es Spring Boot?
03 Anotaciones, beans e inyección
04 Preparar el entorno
05 Crear el proyecto
06 Estructura del proyecto
07 Tu primer endpoint
08 Arquitectura en capas



ELABORADO POR

MARVIN RAFAEL MEZA PINEDA

Facilitador · Centro Nacional de Innovación y Tecnología
Francisco “El Chele” Moreno

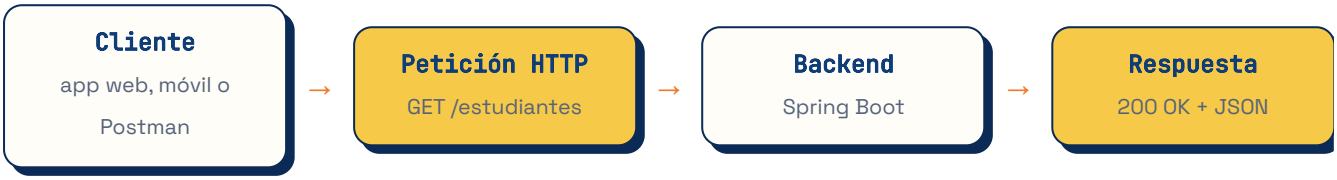
Fundamentos de Desarrollo
Backend con Java
Octubre 2026

CAPÍTULO 01

1 BACKEND, API, HTTP Y JSON

> Hasta ahora nuestros programas vivían en la consola: leían con `Scanner` y respondían con `println`. Un **backend** hace lo mismo, pero sus "preguntas" llegan por internet desde otras aplicaciones.

Cuando abres una app de tu teléfono y ves tu lista de contactos, tus notas o tus mensajes, esa información no vive en el teléfono: la app se la **pide** a un programa que corre en un servidor. Ese programa es el **backend**, y la puerta por la que se le hacen pedidos es su **API**.



- **API** (*Application Programming Interface*): el conjunto de direcciones a las que un cliente puede hacer pedidos. Cada dirección se llama **endpoint**, por ejemplo `/estudiantes`.
- **REST**: un estilo para diseñar APIs sobre HTTP en el que cada dirección representa un **recurso** (estudiantes, cursos, notas) y el **método** HTTP indica qué hacer con él.

Los métodos HTTP

Son los "verbos" de una petición. Con cuatro de ellos se cubren las operaciones básicas sobre datos, conocidas como **CRUD** (crear, leer, actualizar, eliminar):

Método	CRUD	Ejemplo	Significado
`GET`	Leer	`GET /estudiantes`	Dame la lista de estudiantes.
`POST`	Crear	`POST /estudiantes`	Guarda este estudiante nuevo.
`PUT`	Actualizar	`PUT /estudiantes/5`	Reemplaza los datos del estudiante 5.
`DELETE`	Eliminar	`DELETE /estudiantes/5`	Borra al estudiante 5.

Códigos de estado

Toda respuesta incluye un número de tres cifras que resume qué pasó. El primer dígito indica la familia:

Código	Nombre	¿Cuándo aparece?
`200`	OK	Todo salió bien.
`201`	Created	Se creó un recurso nuevo (después de un `POST`).
`400`	Bad Request	El cliente envió datos incorrectos.
`404`	Not Found	La dirección o el recurso no existe.
`500`	Internal Server Error	Algo falló dentro del backend: un error en nuestro código.

JSON: el idioma de las respuestas

Para intercambiar datos, cliente y servidor usan **JSON** (*JavaScript Object Notation*): texto con pares `"clave": valor`. Si comparas con lo que ya sabes de POO, un objeto JSON se parece mucho a un objeto Java con sus atributos:

OBJETO JAVA

```java
Estudiante e = new Estudiante();
e.setId(1L);
e.setNombre("Ana López");
e.setEdad(17);
e.setActivo(true);
```

EL MISMO OBJETO EN JSON

```json
{
  "id": 1,
  "nombre": "Ana López",
  "edad": 17,
  "activo": true
}
```

BUENA NOTICIA

No tendrás que escribir JSON a mano. Spring Boot convierte automáticamente tus objetos Java en JSON al responder, y el JSON que llega en una petición en objetos Java. A esto se le llama **serialización** y **deserialización**.

CAPÍTULO 02

2 ¿QUÉ ES SPRING BOOT?

> **Spring Boot** es una herramienta del ecosistema Java, construida sobre **Spring Framework**, que permite crear aplicaciones backend y APIs listas para ejecutarse en cuestión de minutos.

Para entenderlo hay que distinguir dos piezas. **Spring Framework** es un conjunto enorme de librerías para construir aplicaciones empresariales: web, seguridad, acceso a datos y más. Es muy poderoso, pero antes exigía mucha configuración manual. **Spring Boot** es la capa que se pone encima para que todo eso funcione “de fábrica”.

SPRING TRADICIONAL

- Configuración extensa en archivos XML o clases Java.
- Instalar y configurar aparte un servidor (Tomcat, WildFly).
- Elegir a mano versiones de librerías compatibles entre sí.
- Mucho código repetitivo (*boilerplate*) antes de escribir la primera línea útil.

CON SPRING BOOT

- Configuración automática con valores razonables.
- El servidor viene **dentro** de la aplicación.
- Paquetes de dependencias ya probados juntos.
- En pocos minutos tienes un endpoint respondiendo.

Su filosofía se resume en **"convención sobre configuración"**: si sigues las convenciones (nombres de carpetas, ubicación de archivos, dependencias estándar), Spring Boot asume lo más común y no te pide configurarlo. Solo escribes configuración cuando quieres algo diferente.

Sus cuatro características clave

Característica	Qué hace por ti
Servidor embebido	Incluye un servidor web (**Tomcat** por defecto; también soporta Jetty) dentro de la aplicación. Se ejecuta como un archivo `.jar` con `java -jar`, sin instalar un servidor externo.
Starters	Paquetes de dependencias agrupadas por tema. En lugar de agregar una por una las librerías para web, JSON y servidor, incluyes un solo *starter* web.
Autoconfiguración	Revisa qué librerías tiene tu proyecto y configura sola los componentes necesarios. Por ejemplo, si agregas la base de datos **H2**, crea la conexión automáticamente.
Actuator	Agrega endpoints para vigilar la aplicación en producción: estado de salud (`/actuator/health`), métricas y más.

VERSIÓN DEL CURSO

Usaremos la línea **Spring Boot 4** (la versión estable actual es la **4.1**, lanzada en junio de 2026), que funciona con **Java 17 o superior**, así que es compatible con el **JDK 25** que instalamos en la Lectura 01. Al crear el proyecto, elige siempre la versión estable más reciente: evita las que terminan en `SNAPSHOT` o `M1`, `RC1`, que son versiones de prueba.

CAPÍTULO 03

3 ANOTACIONES, BEANS E INYECCIÓN

> Antes de ver código de Spring necesitas tres ideas nuevas. Con ellas, todo lo demás tendrá sentido.

Las anotaciones (@)

Una **anotación** es una etiqueta que empieza con `@` y se coloca sobre una clase, un método o un atributo. No cambia lo que hace tu código: le da **instrucciones a Spring** sobre cómo tratarlo. Ya conoces una: `@Override`, que usamos en herencia y polimorfismo.

```
@RestController            // "Spring: esta clase atiende peticiones web"
public class HolaController {

    @GetMapping("/hola")   // "Spring: este método responde a GET /hola"
    public String saludar() {
        return "¡Hola desde Spring Boot!";
    }
}
```

Los beans y el contenedor de Spring

En los programas de consola, tú creabas cada objeto con `new`. En Spring Boot, la mayoría de los objetos importantes los crea y administra Spring. A esos objetos se les llama **beans**, y el lugar donde viven es el **contenedor de Spring**.

Al arrancar la aplicación, Spring busca las clases marcadas con anotaciones como `@RestController`, `@Service` o `@Repository`, crea un objeto de cada una y los guarda listos para usarse. A esta idea de "ya no creo yo los objetos, los crea el framework" se le llama **inversión de control** (IoC).

La inyección de dependencias

Si una clase necesita a otra para trabajar, decimos que **depende** de ella. En lugar de crearla con `new`, la clase la pide en su **constructor** y Spring se la entrega ya construida. Eso es la **inyección de dependencias**:

SIN SPRING · YO CREO TODO

```
public class EstudianteController {

  private EstudianteService servicio
      = new EstudianteService();

  // ✗ la clase decide qué objeto
  //   usar y cómo construirlo
}
```

CON SPRING · ME LO INYECTAN

```
public class EstudianteController {

  private final EstudianteService servicio;

  public EstudianteController(
          EstudianteService servicio) {
    this.servicio = servicio;  // ✓ Spring
lo entrega
  }
}
```

¿Por qué es mejor? Porque las clases quedan **desacopladas**: el controlador no necesita saber cómo se construye el servicio. Es la misma idea que vimos con las **interfaces** en la Lectura 04: programar contra "lo que hace" un objeto y no contra "cómo está hecho". Además, en las pruebas se puede entregar un objeto falso sin cambiar el controlador.

BUENA PRÁCTICA

Inyecta siempre por **constructor** y marca el atributo como `final`. Si la clase tiene un solo constructor, Spring lo usa automáticamente: no hace falta ninguna anotación extra. En código antiguo verás `@Autowired` sobre los atributos; hoy se recomienda el constructor.

CAPÍTULO 04

4 PREPARAR EL ENTORNO

> La buena noticia: ya tienes casi todo. Solo faltan unas extensiones de VS Code.

Herramienta	Estado	Detalle
JDK 25	Ya instalado	Lo configuramos en la Lectura 01 con `JAVA_HOME` y `Path`.
Maven	No hace falta instalarlo	Es el gestor de dependencias y construcción del proyecto. Cada proyecto trae el **Maven Wrapper** (`mvnw`), que descarga Maven solo.
Visual Studio Code	Ya instalado	Con el *Extension Pack for Java* de la Lectura 01.
Postman (opcional)	Recomendado	Para probar peticiones `POST`, `PUT` y `DELETE`, que el navegador no hace directamente.

Extensiones de VS Code

Extensión	Publicador	Para qué sirve
Extension Pack for Java	Microsoft	Soporte del lenguaje, autocompletado, depurador, pruebas y Maven. **Ya lo tienes.**
Spring Boot Extension Pack	VMware (Broadcom)	Asistente **Spring Initializr** para crear proyectos, autocompletado en `application.properties` y el panel **Spring Boot Dashboard** para iniciar y detener aplicaciones con un clic.

YA NO HACE FALTA

Guías antiguas recomiendan instalar una extensión aparte para **Lombok** (“Lombok Annotations Support”). Hoy está **obsoleta**: el soporte de Lombok ya viene incluido en el *Extension Pack for Java*. Si la tienes instalada, puedes desinstalarla.

Para instalar la extensión: abre Extensiones con `Ctrl`+`Shift`+`X`, busca **Spring Boot Extension Pack** y pulsa **Instalar**.

CAPÍTULO 05

5 CREAR EL PROYECTO

> Los proyectos Spring Boot se generan con **Spring Initializr**, un asistente que arma la estructura de carpetas, el `pom.xml` y la clase principal por ti.

1. En VS Code abre la paleta de comandos con `Ctrl`+`Shift`+`P` y escribe **Spring Initializr: Create a Maven Project**.
2. Responde las preguntas del asistente con los valores de la tabla de abajo.
3. Elige las dependencias (siguiente tabla), selecciona la carpeta donde se guardará el proyecto y pulsa **Open** cuando VS Code lo ofrezca.

Pregunta	Valor sugerido	Qué significa
Spring Boot version	`4.1.x`	La estable más reciente (sin SNAPSHOT).
Project language	`Java`	También existen Kotlin y Groovy.
Group Id	`ni.cinicaragua`	Identifica a la organización; por convención es un dominio al revés.
Artifact Id	`cursoapi`	El nombre del proyecto, en minúsculas.
Packaging type	`Jar`	Un solo archivo ejecutable con el servidor incluido.
Java version	`25`	La del JDK que tienes instalado.

ALTERNATIVA WEB

Lo mismo se puede hacer en `start.spring.io`: llenas el formulario, pulsas **Generate**, descargas un `.zip`, lo descomprimes y abres la carpeta en VS Code.

Dependencias para empezar

Dependencia	Para qué sirve
Spring Web	Construir APIs REST con Spring MVC. Incluye el servidor **Tomcat** embebido y la conversión automática a JSON. **Es la única indispensable para empezar.**
Spring Boot DevTools	Reinicia la aplicación sola cada vez que guardas un cambio, para no detenerla y arrancarla a mano.
Spring Data JPA	Trabajar con bases de datos relacionales usando objetos Java, sin escribir SQL para las operaciones comunes. Usa **Hibernate** por debajo.
H2 Database	Una base de datos que vive **en memoria**: no hay que instalar nada, ideal para aprender. Se borra al detener la aplicación.
Validation	Validar datos de entrada con anotaciones como `@NotBlank` o `@Min`.
Lombok (opcional)	Genera getters, setters y constructores con anotaciones como `@Data`. En el curso escribiremos primero esos métodos a mano, para que entiendas lo que Lombok hace por ti.

Más adelante cambiaremos H2 por una base de datos real como **PostgreSQL** o **MySQL**: solo hay que agregar su *driver* y cambiar unas líneas de configuración. El resto del código no cambia.

CAPÍTULO 06

6 ESTRUCTURA DEL PROYECTO

> Spring Boot sigue la estructura estándar de Maven. Gracias a la "convención sobre configuración", cada cosa tiene un lugar fijo y Spring sabe dónde buscarla.

```
cursoapi/
├── src/
│   ├── main/
│   │   ├── java/
│   │   │   └── ni/cinicaragua/cursoapi/
│   │   │       ├── CursoapiApplication.java   ← clase principal (punto de arranque)
│   │   │       ├── controller/                ← capa de controladores REST
│   │   │       ├── service/                   ← capa de lógica de negocio
│   │   │       ├── repository/                ← capa de acceso a datos
│   │   │       └── model/                     ← entidades (las tablas)
│   │   └── resources/
│   │       ├── application.properties         ← configuración del proyecto
│   │       ├── static/                        ← archivos estáticos (CSS, JS, imágenes)
│   │       └── templates/                     ← plantillas HTML (si las hubiera)
│   └── test/
│       └── java/                              ← pruebas automáticas
├── mvnw  ·  mvnw.cmd                          ← Maven Wrapper (Linux/Mac · Windows)
└── pom.xml                                    ← dependencias y configuración de Maven
```

OJO CON LOS PAQUETES

Las carpetas `controller`, `service`, `repository` y `model` no las crea el asistente: las creas tú. Deben estar **dentro** del paquete de la clase principal (`ni.cinicaragua.cursoapi`), porque Spring solo busca componentes en ese paquete y sus subpaquetes.

La clase principal

Es el punto de arranque, igual que el `main` de nuestros programas de consola. La diferencia es que su único trabajo es encender Spring Boot:

```
CursoapiApplication.java
package ni.cinicaragua.cursoapi;

import org.springframework.boot.SpringApplication;
import org.springframework.boot.autoconfigure.SpringBootApplication;

@SpringBootApplication
public class CursoapiApplication {

    public static void main(String[] args) {
        SpringApplication.run(CursoapiApplication.class, args);
    }
}
```

La anotación `@SpringBootApplication` equivale a tres anotaciones en una:

Anotación	Qué le indica a Spring
`@SpringBootConfiguration`	Esta clase puede declarar configuración y beans.
`@EnableAutoConfiguration`	Activa la autoconfiguración según las dependencias del proyecto.
`@ComponentScan`	Busca componentes (controladores, servicios, repositorios) en este paquete y sus subpaquetes.

El archivo application.properties

Está en `src/main/resources/` y guarda la configuración: puerto, base de datos, mensajes de registro (*logs*), etc. Gracias a la autoconfiguración, basta con muy pocas líneas:

```
# Nombre de la aplicación
spring.application.name=cursoapi

# Puerto del servidor embebido (8080 es el valor por defecto)
server.port=8080

# Base de datos H2 en memoria
spring.datasource.url=jdbc:h2:mem:cursodb

# Mostrar en consola el SQL que genera Hibernate (útil para aprender)
spring.jpa.show-sql=true
```

MENOS ES MÁS

En guías antiguas verás también el *driver*, el usuario `sa` y el *dialecto* de Hibernate. Con Spring Boot no hace falta: los deduce a partir de la URL y de las dependencias. Escribe solo lo que quieras cambiar.

El archivo pom.xml

Es la "lista de ingredientes" del proyecto: versión de Java, versión de Spring Boot y dependencias. Cuando agregas una dependencia aquí, Maven la descarga automáticamente. Cada dependencia elegida en Initializr aparece como un bloque `<dependency>`, por ejemplo el *starter* web:

```xml
<!-- En Spring Boot 4 el starter web se llama spring-boot-starter-webmvc -->
<dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-webmvc</artifactId>
</dependency>
```

CAPÍTULO 07

7 TU PRIMER ENDPOINT

> Es momento de que la aplicación responda. Creamos un controlador en el paquete `controller` con dos endpoints sencillos.

controller/HolaController.java

```java
package ni.cinicaragua.cursoapi.controller;

import org.springframework.web.bind.annotation.GetMapping;
import org.springframework.web.bind.annotation.PathVariable;
import org.springframework.web.bind.annotation.RestController;

@RestController
public class HolaController {

    // GET http://localhost:8080/hola
    @GetMapping("/hola")
    public String saludar() {
        return "¡Hola desde Spring Boot! CI Nicaragua · Temporada XIV";
    }

    // GET http://localhost:8080/hola/Ana  ->  el nombre viaja en la dirección
    @GetMapping("/hola/{nombre}")
    public String saludarA(@PathVariable String nombre) {
        return "¡Hola, " + nombre + "! Bienvenido al backend.";
    }
}
```

- `@RestController` marca la clase como controlador REST: lo que devuelvan sus métodos se envía como respuesta al cliente.

- `@GetMapping("/hola")` conecta el método con la dirección `/hola` y el método HTTP `GET`. Existen también `@PostMapping`, `@PutMapping` y `@DeleteMapping`.
- `{nombre}` es una parte variable de la dirección; `@PathVariable` la copia en el parámetro del método.

Ejecutar la aplicación

Hay dos formas. En el panel **Spring Boot Dashboard** de VS Code, pulsa el botón ▶ junto a `cursoapi`. O desde la terminal, en la carpeta del proyecto:

```
PS> .\mvnw spring-boot:run          # Windows (PowerShell)
$  ./mvnw spring-boot:run           # Linux o macOS
...
Tomcat started on port 8080 (http) with context path '/'
Started CursoapiApplication in 2.1 seconds
```

Cuando veas **"Tomcat started on port 8080"**, abre el navegador en `http://localhost:8080/hola` y luego en `http://localhost:8080/hola/Ana`. El navegador hace peticiones `GET`, así que es suficiente para probar estos endpoints. Para detener la aplicación pulsa `Ctrl`+`C` en la terminal.

ERRORES FRECUENTES

"Port 8080 was already in use": ya hay otra aplicación usando ese puerto, probablemente una copia anterior de la tuya. Detenla, o cambia `server.port`.
Error 404 (Whitelabel Error Page): la dirección no coincide con ningún `@GetMapping`, o el controlador está fuera del paquete principal y Spring no lo encontró.
La primera ejecución tarda: Maven está descargando las dependencias. Las siguientes serán rápidas.

CAPÍTULO 08

8 ARQUITECTURA EN CAPAS

> En una aplicación real no ponemos todo el código en el controlador. Lo repartimos en **capas**, cada una con una sola responsabilidad. Así el código es más fácil de entender, probar y cambiar.

Una forma sencilla de entenderlo es pensar en un **restaurante**:

CONTROLLER
el mesero
Recibe el pedido del cliente y le entrega la respuesta. No cocina.

SERVICE
el cocinero
Aplica las reglas: qué se puede preparar y cómo. Es el que "piensa".

REPOSITORY
la bodega
Guarda y entrega los ingredientes. Solo sabe almacenar y buscar.

MODEL
los ingredientes
Los datos con los que todos trabajan: un estudiante, un curso.

Capa	Anotación	Responsabilidad

Model	`@Entity`	Representa los datos. Cada clase se corresponde con una tabla de la base de datos y cada atributo con una columna.
Repository	`@Repository` o una interfaz que hereda de `JpaRepository`	Guarda, busca, actualiza y elimina datos (CRUD). No toma decisiones.
Service	`@Service`	Contiene las **reglas de negocio** y validaciones: "un estudiante debe tener 14 años o más".
Controller	`@RestController`	Expone los endpoints HTTP, recibe las peticiones, llama al servicio y devuelve la respuesta.

El recorrido de una petición



1. El cliente envía una petición, por ejemplo `GET /estudiantes`.
2. El **controller** la recibe y llama al método correspondiente del **service**.
3. El **service** aplica las reglas de negocio y usa el **repository** para leer o modificar datos.
4. El **repository** ejecuta la consulta en la base de datos y devuelve objetos **entity**.
5. El **service** procesa el resultado y lo devuelve al **controller**.
6. El **controller** responde al cliente: Spring convierte los objetos a JSON y agrega el código de estado, por ejemplo `200 OK`.

Las peticiones siempre viajan **de arriba hacia abajo** y una capa solo habla con la siguiente: el controlador nunca consulta directamente la base de datos.

Las cuatro capas en código

Este ejemplo completo administra estudiantes. Fíjate cómo aparecen los temas de POO que ya conoces. Para ahorrar espacio se omiten las líneas `package` e `import`; VS Code las agrega por ti con `Ctrl` + `.`

model/Estudiante.java · ENCAPSULAMIENTO

```java
@Entity                                  // esta clase es una tabla
public class Estudiante {

    @Id                                  // llave primaria
    @GeneratedValue(strategy = GenerationType.IDENTITY)  // la BD genera el id
    private Long id;
    private String nombre;
    private int edad;

    public Estudiante() { }              // JPA exige un constructor vacío

    public Long getId() { return id; }
    public String getNombre() { return nombre; }
    public void setNombre(String nombre) { this.nombre = nombre; }
    public int getEdad() { return edad; }
    public void setEdad(int edad) { this.edad = edad; }
}
```

repository/EstudianteRepository.java · INTERFACES Y HERENCIA

```java
public interface EstudianteRepository extends JpaRepository<Estudiante, Long> {
    // ¡Vacía! Al heredar de JpaRepository ya tiene findAll(), findById(),
    // save(), deleteById() y más. Spring crea la implementación por ti.
}
```

service/EstudianteService.java · REGLAS DE NEGOCIO

```java
@Service
public class EstudianteService {

    private final EstudianteRepository repositorio;

    public EstudianteService(EstudianteRepository repositorio) {  // inyección
        this.repositorio = repositorio;
    }

    public List<Estudiante> listar() {
        return repositorio.findAll();
    }

    public Estudiante guardar(Estudiante estudiante) {
        if (estudiante.getEdad() < 14) {
            throw new IllegalArgumentException("El curso requiere 14 años cumplidos");
        }
        return repositorio.save(estudiante);
    }
}
```

controller/EstudianteController.java · ENDPOINTS

```java
@RestController
@RequestMapping("/estudiantes")          // prefijo para todos los métodos
public class EstudianteController {

    private final EstudianteService servicio;

    public EstudianteController(EstudianteService servicio) {    // inyección
        this.servicio = servicio;
    }

    @GetMapping                          // GET  /estudiantes
    public List<Estudiante> listar() {
        return servicio.listar();
    }

    @PostMapping                         // POST /estudiantes
    public Estudiante crear(@RequestBody Estudiante estudiante) {
        return servicio.guardar(estudiante);
    }
}
```

- `JpaRepository<Estudiante, Long>`: entre los signos `< >` se indica con qué entidad trabaja el repositorio y de qué tipo es su llave primaria. A esta sintaxis se le llama **genéricos**; es la misma de `List<Estudiante>`.
- `@RequestBody` le indica a Spring que tome el JSON que viene en la petición y lo convierta en un objeto `Estudiante`.
- La regla de los 14 años vive en el **service**, no en el controlador. Si mañana otro endpoint también registra estudiantes, la regla se respeta sin copiarla.

Con la aplicación corriendo, un `POST /estudiantes` enviado desde Postman con el cuerpo `{"nombre": "Ana López", "edad": 17}` guarda a la estudiante, y un `GET /estudiantes` responde:

```
[
  { "id": 1, "nombre": "Ana López", "edad": 17 }
]
```

Resumen de anotaciones

Anotación	Para qué sirve
`@SpringBootApplication`	Marca la clase principal y activa la autoconfiguración.
`@RestController`	Clase que atiende peticiones HTTP y responde con datos (JSON).
`@RequestMapping`	Prefijo de dirección común para todos los endpoints de un controlador.
`@GetMapping` · `@PostMapping` `@PutMapping` · `@DeleteMapping`	Conectan un método con un método HTTP y una dirección.
`@PathVariable`	Toma un valor de la dirección: `/estudiantes/{id}`.

`@RequestBody`	Convierte el JSON del cuerpo de la petición en un objeto Java.
`@Service` · `@Repository`	Marcan las capas de negocio y de datos como beans de Spring.
`@Entity` · `@Id`	Convierten una clase en tabla e indican su llave primaria.

Referencias

Spring. *Spring Boot Reference Documentation*. docs.spring.io/spring-boot

Spring. *Building a RESTful Web Service* (guía oficial). spring.io/guides/gs/rest-service

Microsoft. *Spring Boot in Visual Studio Code*. code.visualstudio.com/docs/java/java-spring-boot



MARVIN RAFAEL MEZA PINEDA

Facilitador · Fundamentos de Desarrollo Backend con Java

Centro Nacional de Innovación y Tecnología Francisco “El Chele” Moreno · Temporada XIV · 2026